

Exercício Prático 01 — Coletor de Dados da OpenF1 para MongoDB

Persona:

Você é um desenvolvedor Python sênior, especialista em integração de APIs REST e bancos de dados NoSQL, com experiência em engenharia de dados para ingestão confiável e modular.

Objetivo

Criar um **script Python 3** que colete dados da **API OpenF1** e os armazene em um banco **MongoDB**, de forma **modular, configurável e idempotente**, seguindo boas práticas de engenharia de software.

Tecnologias e Bibliotecas

- **Linguagem:** Python 3
 - **Bibliotecas obrigatórias:**
 - requests — chamadas HTTP para a API OpenF1
 - pymongo — integração com MongoDB
 - python-dotenv — para gerenciar variáveis de ambiente (string de conexão e parâmetros configuráveis)
-

Fonte de Dados (API OpenF1)

- **Base URL:** <https://api.openf1.org/v1>
 - **Endpoints obrigatórios:**
 1. /sessions — informações das sessões
 2. /drivers — lista de pilotos de uma sessão
 3. /laps — dados de voltas de uma sessão
-

Destino (Banco de Dados MongoDB)

- **Nome do Banco:** openf1_data
- **Collections:**

- /sessions → sessions
- /drivers → drivers
- /laps → laps

Regras de Armazenamento e Prevenção de Duplicatas

- Usar `update_one(..., upsert=True)` para garantir idempotência.
- Chaves únicas:
 - sessions: `session_key`
 - drivers: combinação de `session_key` + `driver_number`
 - laps: combinação de `session_key` + `driver_number` + `lap_number`

Estrutura e Modularidade do Script

Criar **funções separadas** para:

1. Conexão com MongoDB

- Ler `MONGO_URI` de `.env`
- Retornar o objeto `db`

2. Busca de dados da API

python

CopiarEditar

`fetch_data(endpoint: str, params: dict) -> list`

- Faz GET na API
- Retorna lista de resultados

3. Armazenamento em MongoDB

python

CopiarEditar

`save_to_collection(data: list, collection_name: str, unique_keys: list)`

- Insere/atualiza dados conforme chave única

4. Execução principal (main)

- Buscar dados de uma sessão específica
 - Buscar pilotos dessa sessão
 - Buscar voltas dessa sessão
 - Salvar cada conjunto na collection correspondente
-

Caso de Uso de Demonstração

- Sessão de Corrida (session_key=9159)
 - Grande Prêmio da Itália 2023 em Monza (meeting_key=1219)
-

Saídas Esperadas

- Arquivo **f1_data_collector.py** contendo:
 - Código completo
 - Comentários explicativos em cada função
 - Fluxo principal claro e sequencial
- Arquivo **requirements.txt** contendo:

requests

pymongo

python-dotenv

Critérios de Qualidade

- Código legível e modular
- Sem valores "hardcoded" no meio do código (usar variáveis configuráveis no topo ou .env)
- Tratamento básico de erros (requisições e MongoDB)
- Compatível com múltiplas execuções sem duplicar dados